

Universidad de Oriente	
Ciencia de datos	Clase 03 - Ejemplo 1 - Análisis exploratorio de datos

Para este análisis, se utilizará un conjunto de datos, el cual asignaremos al objeto `calidad_aire` en el código. Este conjunto de datos servirá como base para implementar y demostrar todas las técnicas de análisis exploratorio.

Para la adquisición inicial del conjunto de datos, se ha priorizado la automatización mediante el uso de la función `read_csv()`, la cual permite obtener la información directamente desde el catálogo de datos abiertos. Aunque este método puede incrementar ligeramente el tiempo de ejecución inicial, garantiza la reproducibilidad del proceso y simplifica el flujo de trabajo. Como alternativa, si se prefiere tener un mayor control sobre el proceso o se requiere una ejecución más rápida, existe la opción de descargar previamente el archivo desde el enlace proporcionado y cargarlo desde un directorio local.

```
In [22]: import pandas as pd
import numpy as np
from collections import Counter

calidad_aire = pd.read_csv("calidad-del-aire-datos-historicos-diarios.csv", sep=

In [23]: calidad_aire
```

Out[23]:

	Fecha	CO (mg/m3)	NO (ug/m3)	NO2 (ug/m3)	O3 (ug/m3)	PM10 (ug/m3)	PM25 (ug/m3)	SO2 (ug/m3)	Pr
0	2025-12-21	0.8	12.0	24.0	NaN	5.0	NaN	2.0	
1	2025-12-21	NaN	5.0	8.0	62.0	8.0	2.0	2.0	
2	2025-12-21	NaN	2.0	2.0	86.0	NaN	NaN	1.0	
3	2025-12-21	NaN	1.0	1.0	69.0	9.0	NaN	NaN	
4	2025-12-21	NaN	2.0	3.0	76.0	2.0	NaN	15.0	F
...	...	...	...	...	...	...	...	...	
495861	1997-01-01	NaN	17.0	17.0	57.0	62.0	NaN	28.0	
495862	1997-01-01	1.1	23.0	43.0	291.0	52.0	NaN	6.0	
495863	1997-01-01	NaN	19.0	29.0	29.0	NaN	NaN	14.0	
495864	1997-01-01	NaN	31.0	44.0	9.0	36.0	NaN	37.0	
495865	1997-01-01	NaN	27.0	57.0	29.0	32.0	NaN	8.0	

495866 rows × 13 columns



In [26]: `calidad_aire.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 495866 entries, 0 to 495865
Data columns (total 13 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Fecha                 495866 non-null object
1   CO (mg/m3)            108511 non-null float64
2   NO (ug/m3)            460789 non-null float64
3   NO2 (ug/m3)           459228 non-null float64
4   O3 (ug/m3)            311059 non-null float64
5   PM10 (ug/m3)          386073 non-null float64
6   PM25 (ug/m3)          67532 non-null float64
7   SO2 (ug/m3)           388788 non-null float64
8   Provincia             495866 non-null object
9   Estación              495866 non-null object
10  Latitud               491402 non-null float64
11  Longitud              491402 non-null float64
12  posicion              491402 non-null object
dtypes: float64(9), object(4)
memory usage: 49.2+ MB
```

El dataframe contiene 495,866 registros y 13 variables, de las cuales 9 son numéricas y 4 son de tipo object, con un uso aproximado de memoria de 49.2 MB. La variable Fecha está almacenada como texto, por lo que sería conveniente convertirla a formato de fecha para facilitar análisis temporales. En cuanto a los valores faltantes, las variables Provincia y Estación están completas, mientras que Latitud, Longitud y posicion presentan pocos datos ausentes

```
In [31]: columnas_numericas = calidad_aire.select_dtypes(include=[np.number]).columns
```

```
In [33]: columnas_numericas
```

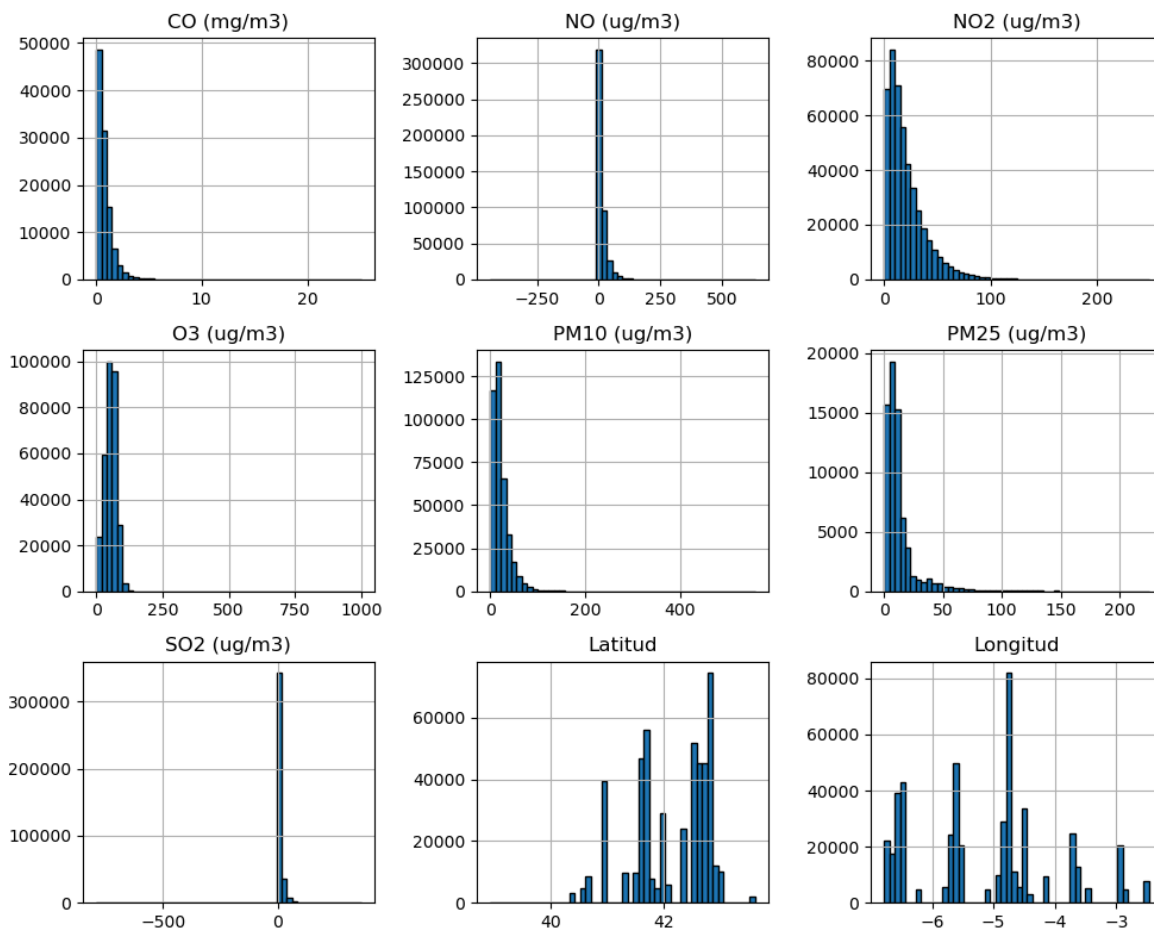
```
Out[33]: Index(['CO (mg/m3)', 'NO (ug/m3)', 'NO2 (ug/m3)', 'O3 (ug/m3)', 'PM10 (ug/m3)',
               'PM25 (ug/m3)', 'SO2 (ug/m3)', 'Latitud', 'Longitud'],
              dtype='object')
```

Representación de histogramas

```
In [36]: import matplotlib.pyplot as plt

plt.figure()
vars_num = calidad_aire[columnas_numericas]
vars_num.hist(figsize=(10,8),bins=50, edgecolor="black")
plt.tight_layout()
plt.show()
```

<Figure size 640x480 with 0 Axes>



Corrección del tipo de variable

```
In [38]: # Ajustar el tipo de la variable Fecha
calidad_aire['Fecha'] = pd.to_datetime(calidad_aire['Fecha'], errors='coerce')

# Ajustar el tipo de la variable Provincia
calidad_aire['Provincia'] = calidad_aire['Provincia'].astype('category')

# Ajustar el tipo de la variable Estación
calidad_aire['Estación'] = calidad_aire['Estación'].astype('category')
```

Identificar valores nulos

```
In [41]: # Devuelve un DataFrame booleano
calidad_aire.isna()
```

Out[41]:

	Fecha	CO (mg/m3)	NO (ug/m3)	NO2 (ug/m3)	O3 (ug/m3)	PM10 (ug/m3)	PM25 (ug/m3)	SO2 (ug/m3)	Pr
0	False	False	False	False	True	False	True	False	
1	False	True	False	False	False	False	False	False	
2	False	True	False	False	False	True	True	False	
3	False	True	False	False	False	False	True	True	
4	False	True	False	False	False	False	True	False	
...	...	...	...	...	...	...	...	...	
495861	False	True	False	False	False	False	True	False	
495862	False	False	False	False	False	False	True	False	
495863	False	True	False	False	False	True	True	False	
495864	False	True	False	False	False	False	True	False	
495865	False	True	False	False	False	False	True	False	

495866 rows × 13 columns



In [43]:

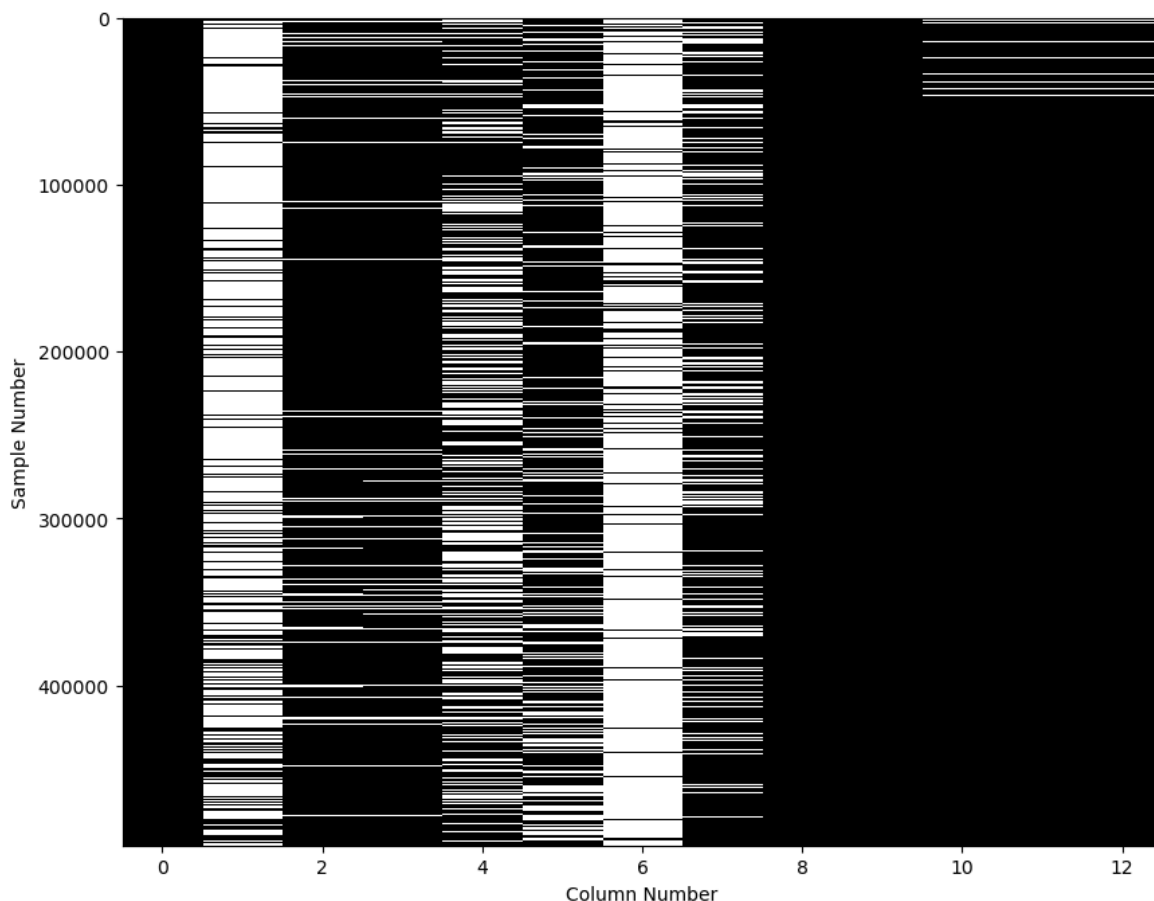
```
# Devuelve el número total de NaN que presenta el DataFrame
print(calidad_aire.isna().sum().sum())
```

1302474

Visualización de valores nulos

In [46]:

```
plt.figure(figsize=(10, 8))
plt.imshow(calidad_aire.isna(), aspect="auto", interpolation="nearest", cmap="gr
plt.xlabel("Column Number")
plt.ylabel("Sample Number");
```



```
In [48]: # Devuelve el % de valores perdidos
print(calidad_aire.isna().mean().mean())
```

0.20205117449534285

Cantidad de valores nulos por variable

```
In [53]: # Detección del número de valores nulos en cada una de las columnas
calidad_aire.isna().sum()
```

```
Out[53]: Fecha                0
CO (mg/m3)          387355
NO (ug/m3)           35077
NO2 (ug/m3)          36638
O3 (ug/m3)          184807
PM10 (ug/m3)         109793
PM25 (ug/m3)         428334
SO2 (ug/m3)          107078
Provincia            0
Estación             0
Latitud              4464
Longitud             4464
posicion             4464
dtype: int64
```

El análisis de valores nulos muestra que las variables Fecha, Provincia y Estación no presentan datos faltantes, mientras que Latitud, Longitud y posicion contienen 4,464 valores nulos cada una, una cantidad relativamente pequeña en comparación con el total de registros. Entre las variables asociadas a contaminantes, PM25 presenta la mayor cantidad de valores faltantes, con 428,334 registros, seguida de CO, con 387,355, y O3,

con 184,807. Por su parte, PM10 y SO2 presentan 109,793 y 107,078 valores nulos, respectivamente, mientras que NO y NO2 muestran una menor cantidad de datos ausentes, con 35,077 y 36,638 registros

```
In [56]: # Detección del % de valores nulos en cada una de las columnas
calidad_aire.isna().mean().round(4)
```

```
Out[56]: Fecha                0.0000
CO (mg/m3)                 0.7812
NO (ug/m3)                 0.0707
NO2 (ug/m3)               0.0739
O3 (ug/m3)                0.3727
PM10 (ug/m3)              0.2214
PM25 (ug/m3)              0.8638
SO2 (ug/m3)               0.2159
Provincia                 0.0000
Estación                 0.0000
Latitud                  0.0090
Longitud                 0.0090
posicion                 0.0090
dtype: float64
```

```
In [58]: # Guardar dataset original
calidad_aire_original = calidad_aire.copy()

# Eliminación de las variables que presentan un % de NaN superior al 50%
calidad_aire = calidad_aire.loc[:, calidad_aire.isna().mean() < 0.5]
```

```
In [60]: calidad_aire
```

Out[60]:

	Fecha	NO (ug/m3)	NO2 (ug/m3)	O3 (ug/m3)	PM10 (ug/m3)	SO2 (ug/m3)	Provincia	Estación	
0	2025-12-21	12.0	24.0	NaN	5.0	2.0	Burgos	Burgos 6	
1	2025-12-21	5.0	8.0	62.0	8.0	2.0	Burgos	Miranda de Ebro 2	42
2	2025-12-21	2.0	2.0	86.0	NaN	1.0	León	Lario	43
3	2025-12-21	1.0	1.0	69.0	9.0	NaN	León	Leon 4	42
4	2025-12-21	2.0	3.0	76.0	2.0	15.0	Palencia	Guardo	42
...	...	...	...	...	...	...	...	...	
495861	1997-01-01	17.0	17.0	57.0	62.0	28.0	León	La Robla	42
495862	1997-01-01	23.0	43.0	291.0	52.0	6.0	Burgos	Burgos1	42
495863	1997-01-01	19.0	29.0	29.0	NaN	14.0	Burgos	Miranda de Ebro2	42
495864	1997-01-01	31.0	44.0	9.0	36.0	37.0	León	Leon2	42
495865	1997-01-01	27.0	57.0	29.0	32.0	8.0	Burgos	Burgos3	41

495866 rows × 11 columns



Reemplazar los valores nulos por la media

```
In [63]: # Seleccionamos las variables numéricas
columnas_numericas = calidad_aire.select_dtypes(include=[np.number]).columns

# Calculamos la media para cada una de las variables numéricas sin tener en cuenta los valores nulos
cols_mean = calidad_aire[columnas_numericas].mean()

# Sustituimos los valores NaN por la media correspondiente a cada variable
calidad_aire[columnas_numericas] = calidad_aire[columnas_numericas].fillna(cols_mean)
```


C:\Users\USER\AppData\Local\Temp\ipykernel_18344\3972122098.py:8: SettingWithCopyWarning:
 Warning:
 A value is trying to be set on a copy of a slice from a DataFrame.
 Try using .loc[row_indexer,col_indexer] = value instead

 See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
 calidad_aire[columnas_numericas] = calidad_aire[columnas_numericas].fillna(cols_mean)

In [65]: calidad_aire

Out[65]:

	Fecha	NO (ug/m3)	NO2 (ug/m3)	O3 (ug/m3)	PM10 (ug/m3)	SO2 (ug/m3)	Provincia	Estación
0	2025-12-21	12.0	24.0	53.250573	5.000000	2.000000	Burgos	Burgo
1	2025-12-21	5.0	8.0	62.000000	8.000000	2.000000	Burgos	Mirand de Ebro
2	2025-12-21	2.0	2.0	86.000000	21.885068	1.000000	León	Lari
3	2025-12-21	1.0	1.0	69.000000	9.000000	8.588948	León	Leon
4	2025-12-21	2.0	3.0	76.000000	2.000000	15.000000	Palencia	Guard
...	...	...	...	...	...	...	...	...
495861	1997-01-01	17.0	17.0	57.000000	62.000000	28.000000	León	La Robl
495862	1997-01-01	23.0	43.0	291.000000	52.000000	6.000000	Burgos	Burgos
495863	1997-01-01	19.0	29.0	29.000000	21.885068	14.000000	Burgos	Mirand d Ebro
495864	1997-01-01	31.0	44.0	9.000000	36.000000	37.000000	León	Leon
495865	1997-01-01	27.0	57.0	29.000000	32.000000	8.000000	Burgos	Burgos

495866 rows × 11 columns



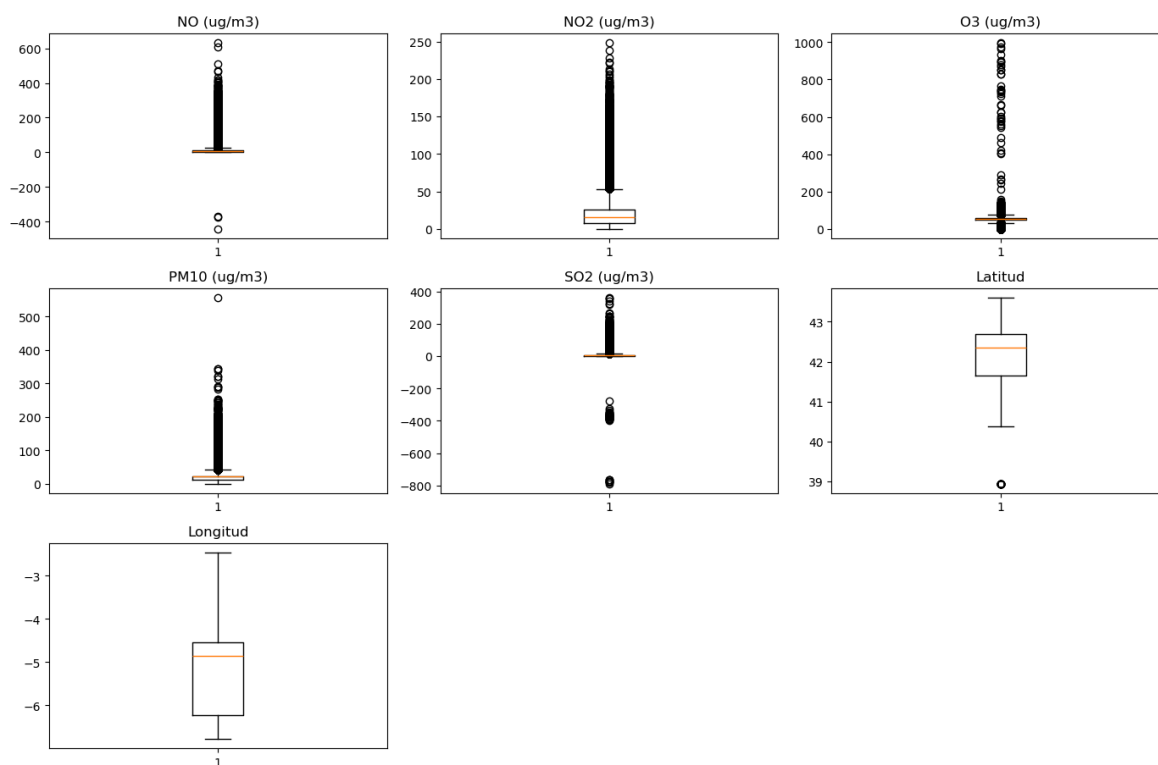
Visualizar valores atípicos

```
In [70]: n = len(columnas_numericas )

plt.figure(figsize=(14, 3*n))
for idx, var in enumerate(columnas_numericas, 1):
    plt.subplot(n, 3, idx)
    plt.boxplot(calidad_aire[var])
```

```
plt.title(var)

plt.tight_layout()
plt.show()
```



Métodos para detectar valores atípicos

```
In [72]: #Método IQR
def outliers_IQR(x):
    Q1 = x.quantile(0.25)
    Q3 = x.quantile(0.75)
    IQR = Q3 - Q1
    li = Q1 - 1.5 * IQR
    ls = Q3 + 1.5 * IQR
    num_outliers = sum((x < li) | (x > ls))
    return num_outliers

#Método de la desviación estándar
def outliers_desv(x):
    mu = x.mean()
    sigma = x.std()
    li = mu - 3 * sigma
    ls = mu + 3 * sigma
    num_outliers = sum((x < li) | (x > ls))
    return num_outliers

#Método de la puntuación z
def outliers_z(df, n, features):
    outlier_list = []

    for column in features:
        data_media = df[column].mean()
        data_std = df[column].std()
        umbral = 3
        z_score = abs( (df[column] - data_media) / data_std )
```

```

        outlier_list_column = df[z_score > umbral].index
        outlier_list.extend(outlier_list_column)

    outlier_list = Counter(outlier_list)
    multiple_outliers = list( k for k, v in outlier_list.items() if v > n )
    df1 = df[z_score > umbral]

    return df1.shape[0]

from scipy.stats import median_abs_deviation
#Método de la puntuación z modificado
def z_scoremod_method(df,n,var):
    outlier_list = []

    for column in var:

        data_media = df[column].mean()
        data_std = df[column].std()
        umbral = 3
        MAD = median_abs_deviation

        mod_z_score = abs(0.6745*(df[column] - data_media)/MAD(df[column]))
        outlier_list_column = df[mod_z_score > umbral].index
        outlier_list.extend(outlier_list_column)

    outlier_list = Counter(outlier_list)
    multiple_outliers = list( k for k, v in outlier_list.items() if v > n )
    df1 = df[mod_z_score > umbral]

    return df1.shape[0]

```

```

In [74]: #Aplicar método IQR
        for vars in columnas_numericas:
            num_outliers = outliers_IQR(calidad_aire[vars])
            print("El número de outliers de la variable %s es %s" %(vars, num_outliers))

```

```

El número de outliers de la variable NO (ug/m3) es 54799
El número de outliers de la variable NO2 (ug/m3) es 27595
El número de outliers de la variable O3 (ug/m3) es 100070
El número de outliers de la variable PM10 (ug/m3) es 41068
El número de outliers de la variable SO2 (ug/m3) es 50379
El número de outliers de la variable Latitud es 36
El número de outliers de la variable Longitud es 0

```

```

In [76]: #Aplicar todos los métodos
        lista_IQR = [outliers_IQR(calidad_aire[vars]) for vars in columnas_numericas]
        lista_desv = [outliers_desv(calidad_aire[vars]) for vars in columnas_numericas]
        lista_z = [outliers_z(calidad_aire,1,[vars]) for vars in columnas_numericas]
        lista_modz = [z_scoremod_method(calidad_aire,1,[vars]) for vars in columnas_nume

        total_outliers = pd.DataFrame({"Variables": columnas_numericas, "Método IQR": li
                                       "Método desviación": lista_desv, "Método puntuaci
                                       "Método puntuación z modificado": lista_modz})

```

```

In [77]: total_outliers

```

Out[77]:

	Variables	Método IQR	Método desviación	Método puntuación Z	Método puntuación z modificado
0	NO (ug/m3)	54799	10237	10237	40415
1	NO2 (ug/m3)	27595	8927	8927	18822
2	O3 (ug/m3)	100070	1210	1210	78485
3	PM10 (ug/m3)	41068	9437	9437	18356
4	SO2 (ug/m3)	50379	8666	8666	33477
5	Latitud	36	36	36	36
6	Longitud	0	0	0	0

El método IQR detecta la mayor cantidad de observaciones atípicas en la mayoría de las variables, destacando O3 con 100,070 casos, seguido de NO con 54,799 y SO2 con 50,379. En contraste, los métodos basados en desviación estándar y puntuación Z producen exactamente los mismos resultados y son mucho más restrictivos, identificando una cantidad considerablemente menor de valores atípicos; por ejemplo, solo 1,210 casos para O3. El método de puntuación Z modificada presenta resultados intermedios, aunque también detecta una cantidad elevada de atípicos en O3 y NO, con 78,485 y 40,415 casos, respectivamente. En las variables geográficas, Latitud presenta únicamente 36 valores atípicos con todos los métodos, mientras que Longitud no registra ninguno

```
In [115... def limites(x):
    Q1 = x.quantile(0.25)
    Q3 = x.quantile(0.75)
    IQR = Q3 - Q1
    li = Q1 - 1.5 * IQR
    ls = Q3 + 1.5 * IQR
    return li, ls
```

Eliminar valores atípicos de O3

```
In [116... li,ls = limites(calidad_aire['O3 (ug/m3)'])
valores_no_out = (calidad_aire['O3 (ug/m3)'] >= li) & (calidad_aire['O3 (ug/m3)']
O3_NoOut = calidad_aire['O3 (ug/m3)'][valores_no_out]
```

Comparación gráfica

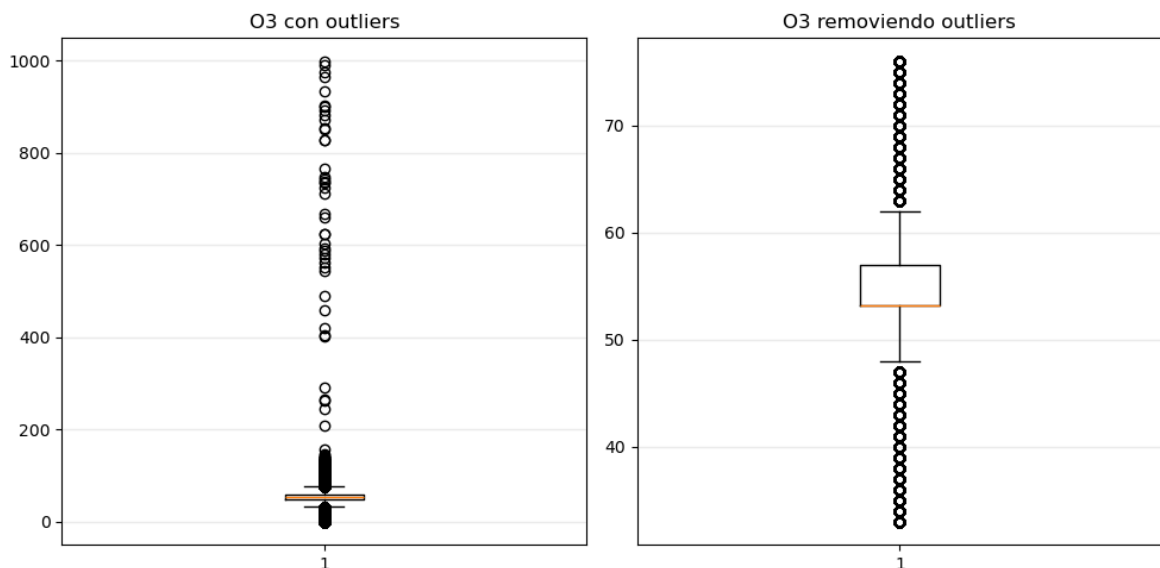
```
In [117... fig, ax = plt.subplots(1, 2, figsize=(10, 5))

ax[0].boxplot(calidad_aire['O3 (ug/m3)'])
ax[0].set_title("O3 con outliers")
ax[0].set_xticks([1])
```

```
ax[0].grid(axis="y", alpha=0.2)

ax[1].boxplot(O3_NoOut )
ax[1].set_title("O3 removiendo outliers")
ax[1].set_xticks([1])
ax[1].grid(axis="y", alpha=0.2)

plt.tight_layout()
plt.show()
```



Detección de valores anómalos

In [123...

```
from sklearn.ensemble import IsolationForest

iso = IsolationForest(contamination=0.05, random_state=2026)

calidad_aire["Anomalias"] = iso.fit_predict(calidad_aire[columnas_numericas])
calidad_aire['Puntaje de anomalía'] = iso.decision_function(calidad_aire[columna
```

C:\Users\USER\AppData\Local\Temp\ipykernel_9420\3308807735.py:5: SettingWithCopyWarning:

A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
calidad_aire["Anomalias"] = iso.fit_predict(calidad_aire[columnas_numericas])
```

In [124...

```
calidad_aire
```

Out[124...

	Fecha	NO (ug/m3)	NO2 (ug/m3)	O3 (ug/m3)	PM10 (ug/m3)	SO2 (ug/m3)	Provincia	Estación
0	2025-12-21	12.0	24.0	53.250573	5.000000	2.000000	Burgos	Burgo
1	2025-12-21	5.0	8.0	62.000000	8.000000	2.000000	Burgos	Mirand de Ebro
2	2025-12-21	2.0	2.0	86.000000	21.885068	1.000000	León	Lari
3	2025-12-21	1.0	1.0	69.000000	9.000000	8.588948	León	Leon
4	2025-12-21	2.0	3.0	76.000000	2.000000	15.000000	Palencia	Guard
...	...	...	...	...	...	...	...	...
495861	1997-01-01	17.0	17.0	57.000000	62.000000	28.000000	León	La Robl
495862	1997-01-01	23.0	43.0	291.000000	52.000000	6.000000	Burgos	Burgos
495863	1997-01-01	19.0	29.0	29.000000	21.885068	14.000000	Burgos	Mirand d Ebro
495864	1997-01-01	31.0	44.0	9.000000	36.000000	37.000000	León	Leon
495865	1997-01-01	27.0	57.0	29.000000	32.000000	8.000000	Burgos	Burgos

495866 rows × 13 columns



In [128...

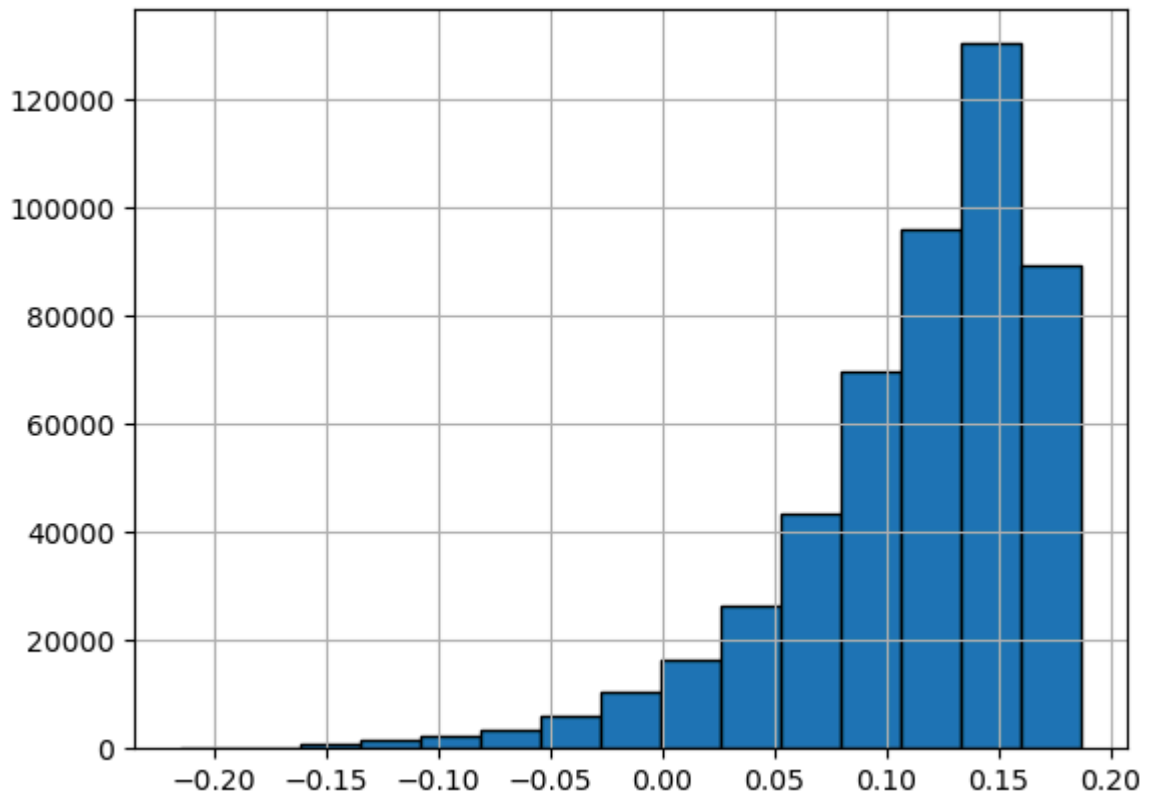
```
calidad_aire['Anomalias'].value_counts()
```

Out[128...

```
Anomalias
1      471072
-1      24794
Name: count, dtype: int64
```

In [130...

```
plt.figure()
calidad_aire['Puntaje de anomalía'].hist(bins=15,edgecolor="black")
plt.show()
```



Matrices de correlación

In [132...

```
import seaborn as sns

# Calculamos la matriz de coeficientes de correlación entre las variables numéri
correlacion = calidad_aire[columnas_numericas].corr()

# Configuración del gráfico de correlación
plt.figure(figsize=(10, 8))

# Gráfico de correlaciones utilizando un mapa de calor
sns.heatmap(correlacion, annot=True, fmt='.4f', linewidths=3, cmap='viridis')

plt.title('Matriz de correlaciones entre variables')
plt.show()
```



La matriz permite concluir que la asociación lineal más importante se encuentra entre \$NO\$ y \$NO_2\$. Además, \$NO\$, \$NO_2\$, \$PM10\$ y \$SO_2\$ muestran algunas correlaciones positivas débiles o moderadas entre sí. Por otro lado, el ozono presenta una tendencia inversa débil respecto a varios contaminantes.

In [134...

```
# Calculamos la matriz de coeficientes de correlación entre las variables numéricas
correlacion_spearman = calidad_aire[columnas_numericas].corr(method='spearman')

# Configuración del gráfico de correlación
plt.figure(figsize=(10, 8))

# Gráfico de correlaciones utilizando un mapa de calor
sns.heatmap(correlacion_spearman, annot=True, fmt='.4f', linewidths=3, cmap='viridis')

plt.title('Correlación de Spearman')
plt.show()
```




La matriz de Spearman confirma el patrón observado con Pearson, NO y NO₂ presentan la relación más fuerte, mientras que \$NO\$, \$NO_2\$, \$PM10\$ y \$SO_2\$ tienden a aumentar conjuntamente en distintos grados. El \$O_3\$ por el contrario, muestra una asociación inversa con NO y NO₂.

In [136...

```
import seaborn as sns

# Calculamos la matriz de coeficientes de correlación entre las variables numéricas
correlacion_kendall = calidad_aire[columnas_numericas].corr(method="kendall")

# Configuración del gráfico de correlación
plt.figure(figsize=(10, 8))

# Gráfico de correlaciones utilizando un mapa de calor
sns.heatmap(correlacion_kendall, annot=True, fmt='.4f', linewidths=3, cmap='viri

plt.title('Correlación de Kendall')
plt.show()
```



La correlación de Kendall confirma los resultados obtenidos anteriormente con Pearson y Spearman. La asociación más importante se encuentra entre \$NO\$ y \$NO_2\$, mientras que \$NO\$, \$NO_2\$, \$PM10\$ y \$SO_2\$ muestran asociaciones positivas entre sí. Por otro lado, el \$O_3\$ presenta una tendencia inversa respecto a \$NO\$ y \$NO_2\$.

In []: